Databasur og SQL (2026)

Uppgáva í databasum

Hansmar Poulsen

Student id: 2004.024

Dato: 27.03.2026

Database assignment

Hansmar Poulsen
Student id: 2004.024
Fróðskaparsetur Føroya

March 2026

Abstract

This report presents the design and implementation of a small banking database in Oracle SQL Developer. The system allows each customer to hold multiple accounts and supports deposits, withdrawals, and transfers. Transactions are initially stored as pending records and are later posted through stored procedures. The implementation further includes views for account statements and customer overviews, stored procedures for transaction posting and monthly interest calculation, a trigger for logging transaction status changes, and a Python console application for interacting with the database.

Contents

1 Problem Formulation **3**

2 Introduction **4**

3 Analysis **5**
3.1 Entity–Relationship Model . . . 5
3.1.1 Entities . . . 6
3.2 Relationships . . . 6
3.3 Constraints . . . 6
3.3.1 Key Constraints . . . 6
3.3.2 Domain Constraints . . . 7
3.3.3 Referential Integrity Constraints . . . 7
3.3.4 Not Null and Check Constraints . . . 7
3.3.5 Functional Dependencies and Normalization . . . 7
3.4 Summary . . . 8

4 Design **9**
4.1 Relational Schema . . . 9
4.1.1 Mapping Entities to Tables . . . 10
4.1.2 Data Types . . . 10
4.2 Journal Entry Workflow . . . 10
4.3 Interest Calculation Design . . . 11
4.4 Planned Database Objects . . . 11

5 Description of the Solution **12**
5.1 Table Creation . . . 12
5.2 Constraints in Practice . . . 13
5.3 Views . . . 13
5.3.1 Account Statement View . . . 13
5.3.2 Customer Overview View (Aggregation) . . . 14
5.3.3 Use of JOIN Operations . . . 14
5.4 Stored Procedures . . . 15
5.4.1 Booking a Transaction . . . 15
5.4.2 Calculating Interest . . . 17
5.5 Trigger . . . 18
5.6 Console Application . . . 18
5.7 Sample Data . . . 18

6 Discussion **19**
6.1 Interest Calculation 19
6.2 Limitations 20
6.3 Possible Improvements 20

7 Conclusion **22**

A Appendix **23**
A.1 GitHub 23
A.2 Use of AI 23
A.3 Console Application Screenshots 24

Chapter 1

Problem Formulation

The objective of this project is to design and implement a relational database system for a small banking application using Oracle SQL Developer. The system must support core banking functionality and ensure that data is stored consistently and processed correctly.

The system must allow a customer to own one or more accounts, and each account must be able to store multiple transactions. It must be possible to perform deposits, withdrawals, and transfers between accounts. In addition, family relationships must be represented such that spouses and parents can access relevant account information for each other and their children.

All financial transactions must follow a two-step process. First, they are entered as pending records in a journal (kassakladda). These transactions are then posted to the actual accounts through stored procedures, ensuring that validation and balance updates are handled in a controlled manner. Monthly interest must also be calculated using stored procedures.

The system must also support the generation of account statements, showing transaction history and current balance for a given account.

These requirements introduce several design challenges. The system must represent relationships between customers, accounts, and family members in a structured way while maintaining referential integrity. Transactions must be handled in a way that ensures consistency, particularly during transfers where multiple accounts are involved. The separation between pending and posted transactions requires a reliable workflow to avoid errors. And interest calculation must be implemented in a way that is consistent across all relevant accounts.

Chapter 2

Introduction

Database systems play a central role in modern applications, particularly in domains where data integrity and consistency are critical. Banking systems are a clear example, as they must handle financial transactions reliably while ensuring that no data is lost or corrupted.

This project explores how a simplified banking system can be designed and implemented using relational database principles. The goal is not only to store data, but also to enforce business rules and support reliable transaction processing through the database itself.

The development process began with identifying the key entities and relationships in the banking domain, such as customers, accounts, transactions, and family connections. These were modeled using an Entity-Relationship diagram, which was then translated into a relational schema. Constraints were applied to enforce data integrity, and additional database objects such as views, stored procedures, and triggers were implemented to support the required functionality.

A central design decision in this project is the use of a two-step transaction workflow, where transactions are first stored as pending records and later posted to accounts. This approach separates data entry from execution and reflects how similar processes are handled in real-world systems.

Chapter 3

Analysis

This chapter analyzes the system requirements and identifies the necessary entities, attributes, and relationships.

3.1 Entity–Relationship Model

The banking system is modeled using the Entity-Relationship model as presented in Section 6.2 of [1]. The purpose of the model is to represent customers, accounts, financial transactions, ownership structures, and family-based access control in a structured and consistent way.

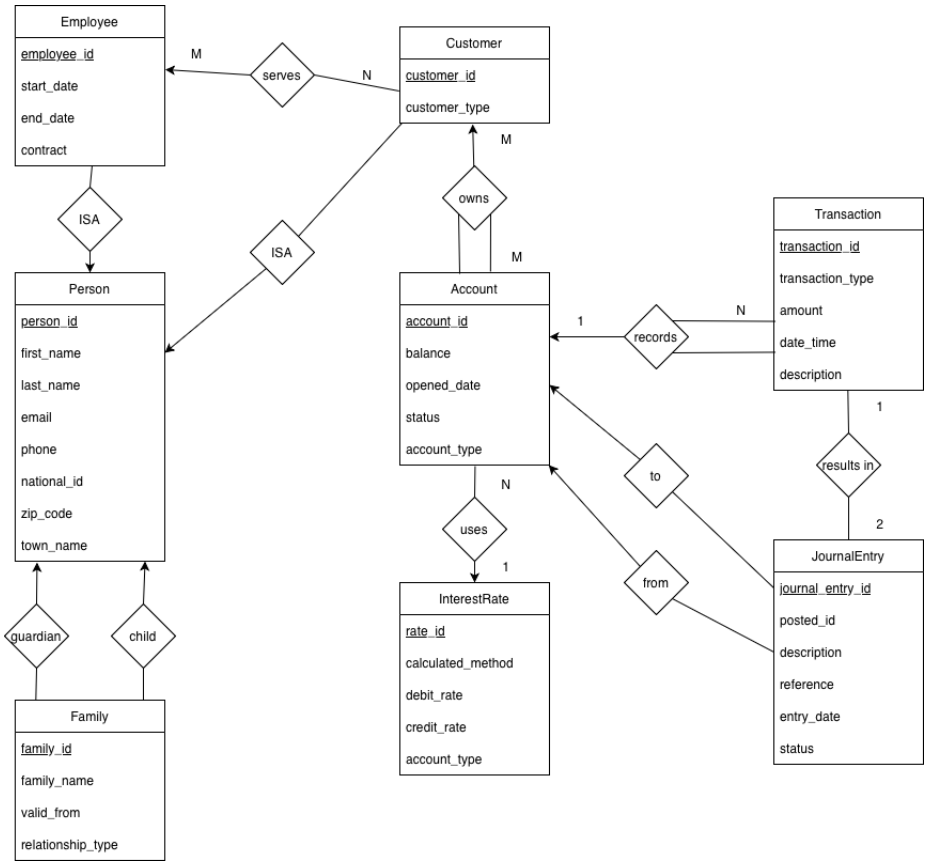

Figure 3.1: Overview of the database structure shown using an Entity–Relationship diagram. A larger version is provided in Appendix A.5.

3.1.1 Entities

The system consists of eight main entities. `Person` stores general personal information shared by both customers and employees, including `person_id`, `first_name`, `last_name`, `email`, `phone`, `national_id`, `zip_code`, and `town_name`. `Customer` and `Employee` are modeled as subtypes of `Person`. `Customer` contains `customer_id` and `customer_type`, while `Employee` contains `employee_id`, `start_date`, `end_date`, and `contract`. This ISA structure avoids duplicating personal information across roles.

`Account` represents the bank accounts with `account_id`, `balance`, `opened_date`, `status`, and `account_type`. `Transaction_` serves a dual role in the system. When a transaction is first entered, it is stored with status `'Pending'` and acts as the kassakladda — the preliminary register where all account movements are recorded before they are officially booked. Once booked, the status changes to `'Posted'`. Its attributes are `transaction_id`, `transaction_type`, `amount`, `date_time`, `description`, and `status`.

`JournalEntry` represents the formal accounting records that are created when a pending transaction is booked. Each booking produces a debit and a credit entry. `JournalEntry` contains `journal_entry_id`, `posted_id`, `description`, `reference`, `entry_date`, and `status`.

`InterestRate` stores interest rules with `rate_id`, `calculated_method`, `debit_rate`, `credit_rate`, and `account_type`. `Family` supports family-based account visibility with `family_id`, `family_name`, `valid_from`, and `relationship_type`.

3.2 Relationships

Person, Customer, and Employee are connected through an ISA (subtype) structure, where common personal information is stored once in Person. The *serves* relationship connects Employee to Customer, representing which employee assists which customer. The *owns* relationship between Customer and Account is many-to-many, supporting joint accounts and shared ownership.

Account and Transaction are connected through the one-to-many *records* relationship. Account and InterestRate are linked through the *uses* relationship, where each account uses one interest rate definition. JournalEntry is connected to Account through two relationships, *from* and *to*, identifying the source and destination accounts. The *results in* relationship between JournalEntry and Transaction shows that one journal entry can produce one or more posted transactions. Family is connected to Person through *guardian* and *child* relationships, defining each person's role within a family.

3.3 Constraints

Constraints are mainly needed to stop invalid banking data from being inserted, for example negative transaction amounts, missing account references, or duplicate national IDs. The constraints described here are based on the categories presented in Section 7.2 of [1].

3.3.1 Key Constraints

Each entity has a primary key: `person_id`, `customer_id`, `employee_id`, `account_id`, `transaction_id`, `journal_entry_id`, `rate_id`, and `family_id`. In addition, `national_id`

in Person functions as a natural candidate key with a unique constraint, since no two persons can share the same national identification number.

3.3.2 Domain Constraints

Domain constraints define permitted values for each attribute. The `balance` attribute in Account must be numeric. The `status` attributes in Account and JournalEntry should only accept limited sets of values such as `'Active'`/`'Closed'` and `'Pending'`/`'Posted'`. The `debit_rate` and `credit_rate` in InterestRate must be numeric and non-negative.

3.3.3 Referential Integrity Constraints

Referential integrity ensures that relationships remain consistent as presented in Section 4.4.5 [1]. Customer and Employee reference Person through foreign keys. The *owns* relationship requires that both the customer and account exist before ownership can be registered. Transaction references Account, and JournalEntry references Account twice (source and destination) as well as Transaction. Account references InterestRate, and the Family relationships require that both the family and person records exist.

3.3.4 Not Null and Check Constraints

Essential attributes such as `first_name`, `last_name`, `national_id`, `balance`, `opened_date`, `amount`, and `date_time` must not be left empty. Check constraints enforce business rules: the `amount` in Transaction must be greater than zero, and interest rates must be non-negative.

3.3.5 Functional Dependencies and Normalization

A functional dependency means that one attribute uniquely determines another. As described by Silberschatz et al., if $A \to B$, then for any two records sharing the same value of A, the value of B must also be the same (Section 7.3.1 of [1]).

The following functional dependencies were identified for the central tables in the system.

Person

$$\text{person_id} \to \text{first_name, last_name, email, phone, national_id, zip_code, town_name}$$
$$\text{national_id} \to \text{person_id, first_name, last_name, email, phone, zip_code, town_name}$$
$$\text{zip_code} \to \text{town_name}$$

Person has two candidate keys: `person_id` (primary key) and `national_id` (unique). The dependency zip_code → town_name is a transitive dependency through the primary key, since person_id → zip_code → town_name. This is discussed further in Section 3.3.5.

Account

$$\text{account_id} \to \text{balance, opened_date, status, account_type, rate_id}$$

The only determinant is the primary key. No transitive or partial dependencies exist.

Transaction_

$$\text{transaction_id} \rightarrow \text{account_id, from_account_id, to_account_id,}$$
$$\text{transaction_type, amount, date_time, description, status}$$

All non-key attributes depend solely on the primary key.

JournalEntry

journal_entry_id → transaction_id, posted_id, description, reference, entry_date, status

All non-key attributes depend solely on the primary key.

InterestRate

$$\text{rate_id} \rightarrow \text{calculated_method, debit_rate, credit_rate, account_type}$$

All non-key attributes depend solely on the primary key.

For Account, Transaction_, JournalEntry, and InterestRate, every non-trivial functional dependency has the primary key as its determinant. This means these tables satisfy both 3NF and BCNF. The only exception is the Person table, which is analyzed below.

Third Normal Form

According to Silberschatz et al., a relation is in third normal form (3NF) if, for every non-trivial functional dependency $A \rightarrow B$, either A is a superkey or B is part of a candidate key (Section 7.3.2 of [1]).

The Person entity contains the transitive dependency:

$$\text{person_id} \rightarrow \text{zip_code} \rightarrow \text{town_name}$$

This violates 3NF because in the dependency zip_code → town_name, `zip_code` is not a superkey of Person, and `town_name` is not part of any candidate key. To achieve strict 3NF, Person could be decomposed into:

- **Person**(<u>person_id</u>, first_name, last_name, email, phone, national_id, zip_code)
- **ZipCode**(<u>zip_code</u>, town_name)

In this project, the decision was made to keep them together because the system is small and the risk of data inconsistency is low. In a larger production system, normalization would be the preferred approach.

3.4 Summary

The analysis showed that the core of the system is the connection between customer, account, and transaction, while family access and interest calculation required additional tables and business rules. Constraints enforce data integrity at multiple levels, and functional dependency analysis confirms that most entities satisfy 3NF, with a documented exception in the Person entity.

Chapter 4

Design

This chapter describes how the E-R model is translated into a relational database design, covering the relational schema, data types, the journal entry workflow, interest calculation logic, and the planned database objects.

4.1 Relational Schema

The E-R model was translated into a relational schema defining the actual table structure. The schema below shows all tables with their columns, data types, primary keys, foreign keys, and unique constraints. It was generated from the implemented database using Oracle SQL Developer's Data Modeler.

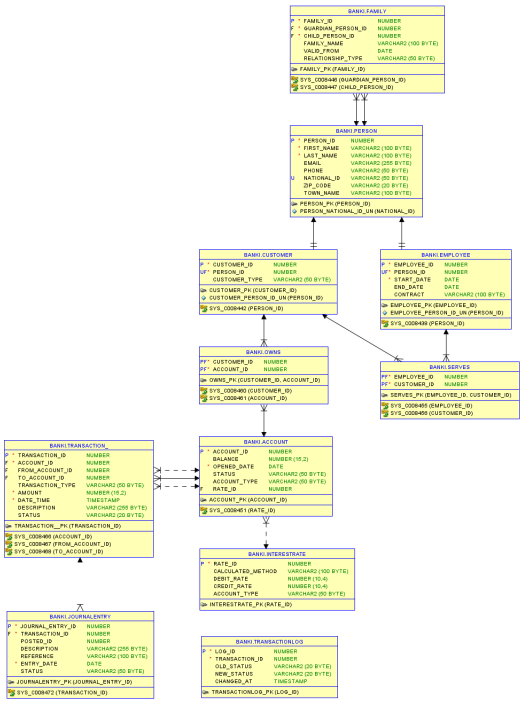

Figure 4.1: Relational schema generated from Oracle SQL Developer. A larger version is provided in Appendix A.6.

4.1.1 Mapping Entities to Tables

Each entity maps directly to a table, following the mapping principles described in Section 6.7 of [1]. The ISA hierarchy is implemented using separate tables: `Customer` and `Employee` each contain a foreign key referencing `Person`, with a `UNIQUE` constraint ensuring a person appears only once per role. This was chosen over a single-table approach, which would result in many null values.

The many-to-many `owns` relationship cannot be represented with foreign keys alone, and is therefore implemented using a junction table called `Owns` with a composite primary key of (`customer_id`, `account_id`). Similarly, the `serves` relationship uses a `Serves` junction table. One-to-many relationships are handled by placing a foreign key in the child table.

The `Transaction_` table contains two foreign keys referencing `Account`: `from_account_id` (source) and `to_account_id` (destination). For deposits the source may be null, for withdrawals the destination may be null, and for transfers both are present. `JournalEntry` references `Transaction_` through a `transaction_id` foreign key to link each journal record to the transaction it was created from. In the E-R model, `JournalEntry` has direct `from` and `to` relationships to `Account`, representing the accounts involved in each journal record. In the implementation, these relationships are handled indirectly through the referenced `Transaction_` record, which already contains the source and destination account IDs. This avoids storing the same account references in two places. The `Family` table uses `guardian_person_id` and `child_person_id`, both referencing `Person`, with `relationship_type` distinguishing between spousal and parent-child connections.

4.1.2 Data Types

Monetary values use `NUMBER(15,2)` for two-decimal precision. Interest rates use `NUMBER(10,4)` to support values like 0.0350. Dates use `DATE`, while transaction timestamps use `TIMESTAMP`. The `national_id` attribute uses `VARCHAR2` rather than `NUMBER` to support leading zeros and non-numeric characters.

4.2 Journal Entry Workflow

A central design decision is the two-step process for handling transactions. All account movements are first entered as pending transactions and then posted through a stored procedure.

In the first step, a new record is created in the Transaction_ table with status `'Pending'`. No account balances are changed. In the second step, a stored procedure reads the pending transaction, validates it (e.g., checking sufficient balance for withdrawals), creates debit and credit journal entries, updates the account balance(s), and sets the status to `'Posted'`. If any error occurs, the entire operation is rolled back.

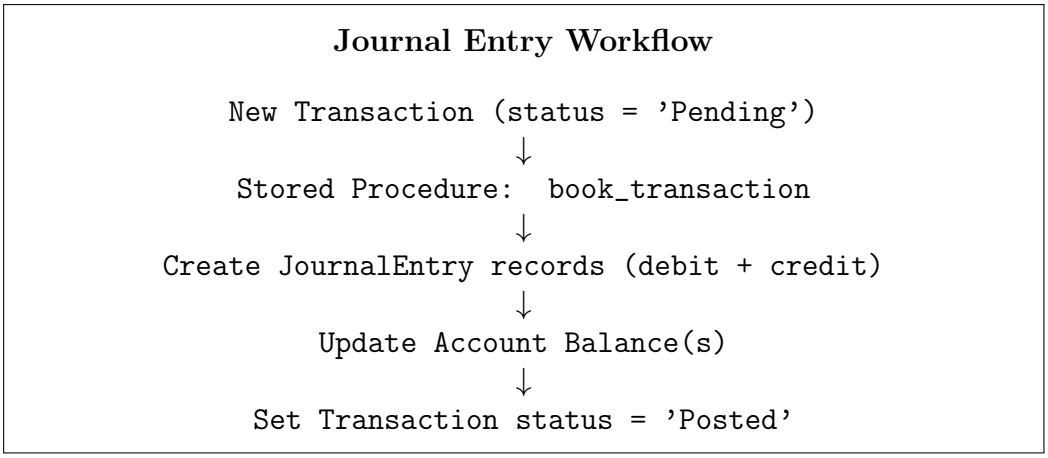

Figure 4.2: Overview of the journal entry posting workflow.

This approach was chosen because it was safer than updating balances immediately. It also made it easier to test transfers and withdrawals before they affected the actual account.

4.3 Interest Calculation Design

Each account is linked to an InterestRate record. When the balance is positive, the credit rate is applied. Monthly interest is calculated as balance × (annual rate / 12). Accounts with a zero credit rate (such as checking accounts) are skipped.

The calculation is handled by a stored procedure that iterates over all active accounts with a positive balance, creates an interest transaction for each, and updates the balance. It is called manually or from the console application by specifying the calculation date.

4.4 Planned Database Objects

Views: One view for account statements, where Account is joined with Transaction_, Owns, Customer, and Person, and another aggregated view showing the number of accounts and total balance per customer (COUNT, SUM).

Triggers: An AFTER UPDATE trigger on Transaction_ to log status changes when transactions are booked.

Stored procedures: A booking procedure to post pending transactions; an interest calculation procedure to process all active accounts; a helper procedure to book all pending transactions at once.

Chapter 5

Description of the Solution

This chapter describes the implementation of the banking database system in Oracle SQL Developer. Key code excerpts are included; full listings are in the Appendix A.1.

5.1 Table Creation

The database consists of eleven tables. The ten core tables from the E-R model are Person, Employee, Customer, Family, InterestRate, Account, Owns, Serves, Transaction_, and JournalEntry. One supporting table, TransactionLog, was added to support the trigger that logs booking activity. TransactionLog was added during implementation as a logging/audit table and is intentionally excluded from the conceptual E-R model.

The table name `Transaction_` uses a trailing underscore because `TRANSACTION` is a reserved word in Oracle SQL.

The most important CREATE TABLE statements are shown below. The Person table stores personal information and enforces a unique constraint on `national_id`:

```
CREATE TABLE Person (
    person_id       NUMBER PRIMARY KEY,
    first_name      VARCHAR2(100) NOT NULL,
    last_name       VARCHAR2(100) NOT NULL,
    email           VARCHAR2(255),
    phone           VARCHAR2(50),
    national_id     VARCHAR2(50) UNIQUE,
    zip_code        VARCHAR2(20),
    town_name       VARCHAR2(100)
);
```

The ISA hierarchy is implemented with foreign keys and UNIQUE constraints on `person_id`:

```
CREATE TABLE Customer (
    customer_id     NUMBER PRIMARY KEY,
    person_id       NUMBER NOT NULL UNIQUE,
    customer_type   VARCHAR2(50),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
);
```

The Transaction_ table includes `from_account_id` and `to_account_id` for source and destination, and a `status` column for the journal entry workflow:

```
CREATE TABLE Transaction_ (
    transaction_id      NUMBER PRIMARY KEY,
    account_id          NUMBER NOT NULL,
    from_account_id     NUMBER,
    to_account_id       NUMBER,
    transaction_type    VARCHAR2(50),
    amount              NUMBER(15, 2) NOT NULL CHECK (amount > 0),
    date_time           TIMESTAMP NOT NULL,
    description         VARCHAR2(255),
    status              VARCHAR2(20) DEFAULT 'Pending',
    FOREIGN KEY (account_id)      REFERENCES Account(account_id),
    FOREIGN KEY (from_account_id) REFERENCES Account(account_id),
    FOREIGN KEY (to_account_id)   REFERENCES Account(account_id)
);
```

The remaining tables were created in the same way, with primary keys, foreign keys, and constraints based on the relationships from the E-R diagram. Three sequences (seq_transaction, seq_journal, seq_log) were created for concurrency-safe ID generation. Full SQL code is included in the appendix/GitHub repository.

5.2 Constraints in Practice

The constraints described in the analysis are enforced through primary keys, foreign keys, unique constraints, and NOT NULL constraints in the CREATE TABLE statements. Domain constraints are partially enforced through data types such as `NUMBER(15,2)` for monetary values. Additional business rule validation, such as checking that the balance is sufficient for a withdrawal, is handled within the stored procedures.

5.3 Views

Views provide virtual tables derived from queries on base tables as presented in Section 4.2 of [1]. Two views are implemented. The views rely heavily on JOIN operations to combine data from multiple tables, which is a natural consequence of the normalized database design.

5.3.1 Account Statement View

The `v_account_statement` view joins five tables to provide a complete account statement. A chain of inner joins follows the path from Account through Owns to Customer and Person to retrieve the owner's name. A LEFT JOIN is used for Transaction_ so that accounts with no transactions are still included.

```
CREATE OR REPLACE VIEW v_account_statement AS
SELECT
    a.account_id,
```

```
    p.first_name || ' ' || p.last_name AS eigari,
    a.account_type, t.date_time,
    t.transaction_type, t.description, t.amount,
    t.status, a.balance AS current_balance
FROM Account a
JOIN Owns o ON a.account_id = o.account_id
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Person p ON c.person_id = p.person_id
LEFT JOIN Transaction_ t ON (
    (t.transaction_type IN ('Deposit', 'Withdrawal', 'Interest')
        AND t.account_id = a.account_id)
    OR
    (t.transaction_type = 'Transfer'
        AND (t.from_account_id = a.account_id
             OR t.to_account_id = a.account_id))
)
ORDER BY a.account_id, t.date_time;
```

5.3.2 Customer Overview View (Aggregation)

The `v_customer_overview` view uses `COUNT` and `SUM` to show each customer's total number of accounts and total balance. This satisfies the requirement for views with aggregation.

```
CREATE OR REPLACE VIEW v_customer_overview AS
SELECT
    c.customer_id,
    p.first_name || ' ' || p.last_name AS kundinavn,
    c.customer_type,
    COUNT(a.account_id) AS antal_konti,
    SUM(a.balance) AS total_saldo
FROM Customer c
JOIN Person p ON c.person_id = p.person_id
JOIN Owns o ON c.customer_id = o.customer_id
JOIN Account a ON o.account_id = a.account_id
GROUP BY c.customer_id, p.first_name, p.last_name,
         c.customer_type;
```

5.3.3 Use of JOIN Operations

Throughout the system, three types of joins are used. **Inner joins** (JOIN) are used when both sides of the relationship must exist, which is the most common case. **Left joins** (LEFT JOIN) are used when the related data is optional, such as in the account statement view where accounts with no transactions should still appear. A **self-join** is used in the family view query in the console application, where the Person table is joined twice with different aliases to retrieve both the guardian's and the child's name:

```
FROM Family f
JOIN Person g  ON f.guardian_person_id = g.person_id
JOIN Person ch ON f.child_person_id = ch.person_id
```

5.4 Stored Procedures

Two main stored procedures implement the core business logic.

5.4.1 Booking a Transaction

The `book_transaction` procedure takes a transaction ID and an optional `p_auto_commit` parameter as input and: (1) fetches the pending transaction using `%ROWTYPE`, (2) verifies it has status `'Pending'`, (3) checks that the source account has sufficient balance for withdrawals and transfers, (4) creates debit and credit journal entries using sequences for ID generation, (5) updates the account balance(s) based on the transaction type, and (6) sets the transaction status to `'Posted'`. If any error occurs, the procedure rolls back and raises a meaningful error message.

A second procedure, `book_all_pending`, uses a cursor to iterate over all pending transactions and calls `book_transaction` for each one with `p_auto_commit => FALSE`. This ensures that no individual transaction is committed separately — a single `COMMIT` is issued after the entire loop, so that if any transaction fails, the whole batch is rolled back.

```
CREATE OR REPLACE PROCEDURE book_transaction (
    p_transaction_id IN NUMBER,
    p_auto_commit    IN BOOLEAN DEFAULT TRUE
) AS
    v_rec        Transaction_%ROWTYPE;
    v_balance    NUMBER(15,2);
    e_insufficient_funds EXCEPTION;
BEGIN
    SELECT * INTO v_rec FROM Transaction_
    WHERE transaction_id = p_transaction_id;

    IF v_rec.status != 'Pending' THEN
        RETURN;
    END IF;

    -- Check sufficient balance
    IF v_rec.transaction_type IN ('Withdrawal', 'Transfer') THEN
        SELECT balance INTO v_balance FROM Account
        WHERE account_id = v_rec.from_account_id;

        IF v_balance < v_rec.amount THEN
            RAISE e_insufficient_funds;
        END IF;
    END IF;

    -- Create debit and credit journal entries
    INSERT INTO JournalEntry ...
    VALUES (seq_journal.NEXTVAL, p_transaction_id, ...);
    INSERT INTO JournalEntry ...
    VALUES (seq_journal.NEXTVAL, p_transaction_id, ...);
```

```
    -- Update balances based on transaction type
    IF v_rec.transaction_type = 'Deposit' THEN
        UPDATE Account SET balance = balance + v_rec.amount
        WHERE account_id = v_rec.to_account_id;
    ELSIF v_rec.transaction_type = 'Withdrawal' THEN
        UPDATE Account SET balance = balance - v_rec.amount
        WHERE account_id = v_rec.from_account_id;
    ELSIF v_rec.transaction_type = 'Transfer' THEN
        UPDATE Account SET balance = balance - v_rec.amount
        WHERE account_id = v_rec.from_account_id;
        UPDATE Account SET balance = balance + v_rec.amount
        WHERE account_id = v_rec.to_account_id;
    END IF;

    UPDATE Transaction_ SET status = 'Posted'
    WHERE transaction_id = p_transaction_id;

    IF p_auto_commit THEN
        COMMIT;
    END IF;

EXCEPTION
    WHEN e_insufficient_funds THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001,
            'Insufficient balance on account '
            || v_rec.from_account_id);
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20002,
            'Transaction ' || p_transaction_id
            || ' not found.');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END book_transaction;
```

The `book_all_pending` procedure calls `book_transaction` with `p_auto_commit => FALSE` to defer the commit:

```
CREATE OR REPLACE PROCEDURE book_all_pending
AS
BEGIN
    FOR rec IN (
        SELECT transaction_id FROM Transaction_
        WHERE status = 'Pending'
        ORDER BY date_time
    ) LOOP
```

```
        book_transaction(rec.transaction_id,
            p_auto_commit => FALSE);
    END LOOP;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END book_all_pending;
```

5.4.2 Calculating Interest

The `calculate_interest` procedure takes a date as input and iterates over all active accounts with a positive balance using a cursor. For each account, it looks up the credit rate, calculates monthly interest as balance × (annual rate / 12), creates an interest transaction, and updates the balance. Accounts with a zero credit rate are skipped.

```
CREATE OR REPLACE PROCEDURE calculate_interest (
    p_calc_date IN DATE
) AS
    v_interest  NUMBER(15,2);
    v_txn_id    NUMBER;
BEGIN
    FOR rec IN (
        SELECT a.account_id, a.balance, ir.credit_rate
        FROM Account a
        JOIN InterestRate ir ON a.rate_id = ir.rate_id
        WHERE a.status = 'Active' AND a.balance > 0
    ) LOOP
        v_interest := ROUND(rec.balance
                     * (rec.credit_rate / 12), 2);
        IF v_interest = 0 THEN CONTINUE; END IF;

        v_txn_id := seq_transaction.NEXTVAL;

        INSERT INTO Transaction_ (...)
        VALUES (v_txn_id, rec.account_id, ...
            'Interest', v_interest, p_calc_date,
            'Renturokning fyri '
            || TO_CHAR(p_calc_date, 'MM/YYYY'),
            'Posted');

        UPDATE Account
        SET balance = balance + v_interest
        WHERE account_id = rec.account_id;
    END LOOP;
    COMMIT;
```

```
END;
```

To run interest calculation for a given month:

```
EXEC calculate_interest(DATE '2026-03-31');
```

5.5 Trigger

A trigger is a statement that the database executes automatically as a side effect of a modification, as presented in Section 5.3 of [1]. The system uses an AFTER UPDATE trigger on the Transaction_ table that logs every status change to a TransactionLog table. When a transaction is booked and its status changes from `'Pending'` to `'Posted'`, the trigger automatically records the old status, new status, transaction ID, and a timestamp.

```
CREATE OR REPLACE TRIGGER trg_transaction_status
AFTER UPDATE OF status ON Transaction_
FOR EACH ROW
BEGIN
    INSERT INTO TransactionLog
    (log_id, transaction_id, old_status, new_status)
    VALUES (
        seq_log.NEXTVAL,
        :OLD.transaction_id,
        :OLD.status,
        :NEW.status
    );
END;
```

This provides an audit trail of all booking activity without requiring changes to the stored procedures themselves.

5.6 Console Application

A Python console application provides the user interface. It connects to the Oracle database using the `oracledb` library and presents a text-based menu in Faroese. The application supports viewing customers, accounts, and family relationships; entering deposits, withdrawals, and transfers as pending transactions; booking single or all pending transactions via the stored procedures; viewing account statements; and accessing family members' accounts. The application does not modify account balances directly — all balance updates are handled by the stored procedures.

5.7 Sample Data

The system was tested with eight persons, three employees, five customers, and seven accounts. The Petersen family demonstrates the family access feature: Jógvan and Katrin are married, and Brandur is their child. Three interest rate definitions were created: savings (1% credit, 5% debit), checking (0% credit, 12% debit), and business (2% credit, 3.5% debit). Eight sample transactions covering all three types were included.

Chapter 6

Discussion

This chapter reflects on the design decisions, challenges encountered during development, limitations, and possible improvements.

6.1 Interest Calculation

The interest calculation uses a stored procedure that is called manually with a date parameter. The procedure iterates over all active accounts with a positive balance, looks up the credit rate from the `InterestRate` table, and calculates monthly interest as balance $\times$ (annual rate/12).

In this project the procedure uses the current balance at the time of execution rather than the average daily balance. In real banking, the average daily balance reflects how much money was in the account across the entire period.

To illustrate the difference, consider a savings account with a 1% annual credit rate. If a customer has 100,000 kr for 29 days and then withdraws 90,000 kr on the last day of the month, the current implementation would calculate interest on the remaining 10,000 kr, giving:

$$10{,}000 \times \frac{0.01}{12} = 8.33 \text{ kr}$$

Using the average daily balance formula:

$$\text{ADB} = \frac{\sum(B_i \times d_i)}{D} = \frac{100{,}000 \times 29 + 10{,}000 \times 1}{30} = 97{,}000 \text{ kr}$$

The interest would then be:

$$97{,}000 \times \frac{0.01}{12} = 80.83 \text{ kr}$$

The customer loses roughly 72 kr in this example, and the difference grows with higher balances or higher rates.

Implementing average daily balance would require computing each account's balance history from the `Transaction_` table for the given month, grouping by the number of days at each balance level. Each transaction has a timestamp, so it could be calculated, but I did not have time for it.

6.2 Limitations

The system has several limitations that would need to be addressed in a production environment.

There is no authentication or role-based access control in the console application. Any user can view any account, book any transaction, or run the interest calculation simply by entering an account or transaction ID. In a real system, the `Serves` relationship between `Employee` and `Customer` could be used to restrict which employees are allowed to access which customers' data, but this is not enforced in the current implementation.

There is also a potential concurrency issue in the `book_transaction` procedure. The procedure first reads the transaction status and account balance with a regular `SELECT`, and then updates the balance in a separate statement. If two users attempted to book the same transaction or withdraw from the same account simultaneously, there is a small window where both could read the same balance before either update is applied. Using `SELECT ... FOR UPDATE` would lock the relevant rows during the procedure and prevent this race condition. For the scope of this project, where only a single user interacts with the database through the console application, this was not a practical concern, but it would be necessary in a multi-user environment.

Finally, the system does not track which employee performed each action. The `TransactionLog` table records status changes with a timestamp, but does not store who initiated the booking. Adding an `employee_id` column to the log table would provide a more complete audit trail.

6.3 Possible Improvements

If the system were to be developed further, some improvements could be done.

As noted in the limitations, any user can currently perform any action. A practical solution would be to add role-based control with a `Role` table with columns such as `role_id` and `role_name` (for example `'Teller'`, `'Manager'`, `'Admin'`), and link it to the `Employee` table through a foreign key. Each role would define which operations the employee is allowed to perform — a teller might only enter transactions into the JournalEntry, while a manager could book them. This mirrors how Oracle itself handles access control through the `GRANT` and `REVOKE` statements described in Section 4.7.1 of [1], where privileges on specific tables or procedures can be assigned to individual users or roles. In a larger system, the application-level roles and the database-level privileges would work together to enforce security at both layers.

Right now the procedure has to be called manually from the console application, which means someone has to remember to do it. Automating the monthly interest calculation is something Oracle provides with `DBMS_SCHEDULER`, a built-in package that can run stored procedures on a schedule. A job could be configured to call `calculate_interest` on the last day of each month automatically. This would remove the dependency on manual execution and reduce the risk of missed or duplicate interest runs. Combined with the average daily balance approach discussed in Section 6.2, this would bring the interest calculation much closer to how it works in a real bank.

Replacing the console application with a web-based interface would be a nice feature. The current text-based menu works for testing and demonstration, but it is not practical for real use. A web application built with something like Flask or Django could present forms for entering transactions, display account statements in a readable format, and

enforce login-based access tied to the role system described above. The database layer and stored procedures would remain the same — only the interface would change, which shows one of the advantages of keeping business logic in the database rather than in the application.

The `national_id` attribute in the Person table is currently stored as a string with a uniqueness constraint, but no validation is performed on the value itself. Faroese personal identification numbers (P-tal) follow the structure: ten digits in the format `DDMMYY-SSSC`, where the first six digits encode the date of birth, the next three form a sequence number, and the final digit is a check digit. The check digit is calculated using a weighted modulo-11 algorithm. Each of the ten digits is multiplied by a corresponding weight from the sequence $(4, 3, 2, 7, 6, 5, 4, 3, 2, 1)$, the products are summed, and the result must be divisible by 11. A PL/SQL function could perform this validation and be called from a `BEFORE INSERT` trigger on the Person table, rejecting any record where the `national_id` does not pass the check. This would move validation from the application layer into the database itself, which is generally preferable since it ensures consistency regardless of which client inserts the data.

Similarly, a function for generating and validating bank account numbers could be implemented. Faroese bank accounts follow a registration number and account number format, where the registration number identifies the bank. A PL/SQL function could assign account numbers automatically when new accounts are created, enforcing a consistent format and potentially including a check digit for error detection. This would replace the current approach where `account_id` is simply a manually chosen number.

Chapter 7

Conclusion

This project presents the design and implementation of a small banking system using a relational database in Oracle SQL Developer. The system supports core banking operations such as deposits, withdrawals, transfers, account statements, and monthly interest calculation. All implemented through a combination of tables, constraints, views, triggers, and stored procedures.

The project shows how theoretical concepts from database systems, including entity-relationship modeling, normalization, functional dependencies, and transaction management, can be applied in practice. The use of a two-step transaction workflow with pending records and stored procedure booking demonstrates the importance of maintaining data consistency and control over critical operations.

I have learned from this project that database design is not only about structuring data, but also about enforcing business rules and ensuring reliable system behavior.

Appendix A

Appendix

SQL code and additional technical documentation are included here.

A.1 GitHub

https://github.com/hansmar/database_sql_report_appendix
All SQL code and Python code can be read at this url.

A.2 Use of AI

During this project, Claude Opus 4.6 (Anthropic) was used as a support tool in the project development. Claude was used to help structure sections of the report, and helped with LaTeX syntax and formatting of the document. Claude was used for general debugging and troubleshooting during development.

A.3 Console Application Screenshots

```
PS C:\Users\Administrator\Documents\Python> & C:\Users\Administrator\
Documents/Python/banking_console_python.py

============================================================
  Bankaskipan
============================================================
  Samband við databasan!

============================================================
  HØVUÐSMENU
============================================================

  -- Yvirlit --
  1. Vís kundar
  2. Vís kontoir
  3. Vís familju samband
  4. Kontoavrit (account statement)
  5. Familju kontoir (spouse/child access)

  -- Kassakladda (tasta inn) --
  6. Innlegg (Deposit)
  7. Úttøka (Withdrawal)
  8. Flyting (Transfer)
  9. Vís óbókaðar transaktionir

  -- Bóking --
  10. Bóka eina transaktión
  11. Bóka allar óbókaðar
  12. Vís journal entries
  13. Renturokning (interest calculation)

  0.  Enda
------------------------------------------------------------
  Vel:
```

Figure A.1: Main menu of the console application.

```
============================================================
  HØVUÐSMENU
============================================================

  -- Yvirlit --
  1. Vís kundar
  2. Vís kontoir
  3. Vís familju samband
  4. Kontoavrit (account statement)
  5. Familju kontoir (spouse/child access)

  -- Kassakladda (tasta inn) --
  6. Innlegg (Deposit)
  7. Úttøka (Withdrawal)
  8. Flyting (Transfer)
  9. Vís óbókaðar transaktionir

  -- Bóking --
  10. Bóka eina transaktión
  11. Bóka allar óbókaðar
  12. Vís journal entries
  13. Renturokning (interest calculation)

  0.  Enda
------------------------------------------------------------
  Vel: 6

============================================================
  Kassakladda - Innlegg (Deposit)
============================================================
  Konto ID: 1004
  Upphædd: 500
  Lýsing: Nýggjar buksir

  Innlegg av 500.00 kr skrásett á kassakladdu. (TXN-106)
  Status: Pending - bóka fyri at gera galdandi.
============================================================
```

Figure A.2: Entering a deposit into the Transaction_ table.

```
  2. Vís kontoir
  3. Vís familju samband
  4. Kontoavrit (account statement)
  5. Familju kontoir (spouse/child access)

  -- Kassakladda (tasta inn) --
  6. Innlegg (Deposit)
  7. Úttøka (Withdrawal)
  8. Flyting (Transfer)
  9. Vís óbókaðar transaktionir

  -- Bóking --
  10. Bóka eina transaktión
  11. Bóka allar óbókaðar
  12. Vís journal entries
  13. Renturokning (interest calculation)

  0.  Enda
-------------------------------------------------------
  Vel: 4

=======================================================
  Kontoavrit
=======================================================
  Skriva konto ID: 1001

  Konto:  1001  (Checking)
  Eigari: Jógvan Petersen
  Status: Active

  Dato/Tíð            Slag      Lýsing                    Frá   Til   Upphædd  Status
  ---------------------------------------------------------------------------------------
  2026-03-01 08:15:00  Deposit   Lønargreiðsla             None  1001  15000.0  Pending
  2026-03-02 10:30:00  Transfer  Flutt til sparikonto       1001  1002  5000.0   Pending
  2026-03-05 14:20:00  Transfer  Rúmleiga                  1005  1001  3500.0   Pending
  2026-03-07 12:30:00  Transfer  Mánaðarpengar til Brandur  1001  1004  1000.0   Pending

  Núverandi saldo: 45,000.00 kr
```

Figure A.3: Showing an account statement.

```
============================================================
  HØVUÐSMENU
============================================================

  -- Yvirlit --
  1. Vís kundar
  2. Vís kontoir
  3. Vís familju samband
  4. Kontoavrit (account statement)
  5. Familju kontoir (spouse/child access)

  -- Kassakladda (tasta inn) --
  6. Innlegg (Deposit)
  7. Úttøka (Withdrawal)
  8. Flyting (Transfer)
  9. Vís óbókaðar transaktionir

  -- Bóking --
  10. Bóka eina transaktión
  11. Bóka allar óbókaðar
  12. Vís journal entries
  13. Renturokning (interest calculation)

  0.  Enda
------------------------------------------------------------
  Vel: 11

============================================================
  Bóka allar óbókaðar transaktionir
============================================================

  9 óbókaðar transaktionir funnir.
  Bóka allar? (j/n): j

  Allar 9 transaktionir eru bókaðar!
============================================================
```

Figure A.4: Image of book all pending transactions.

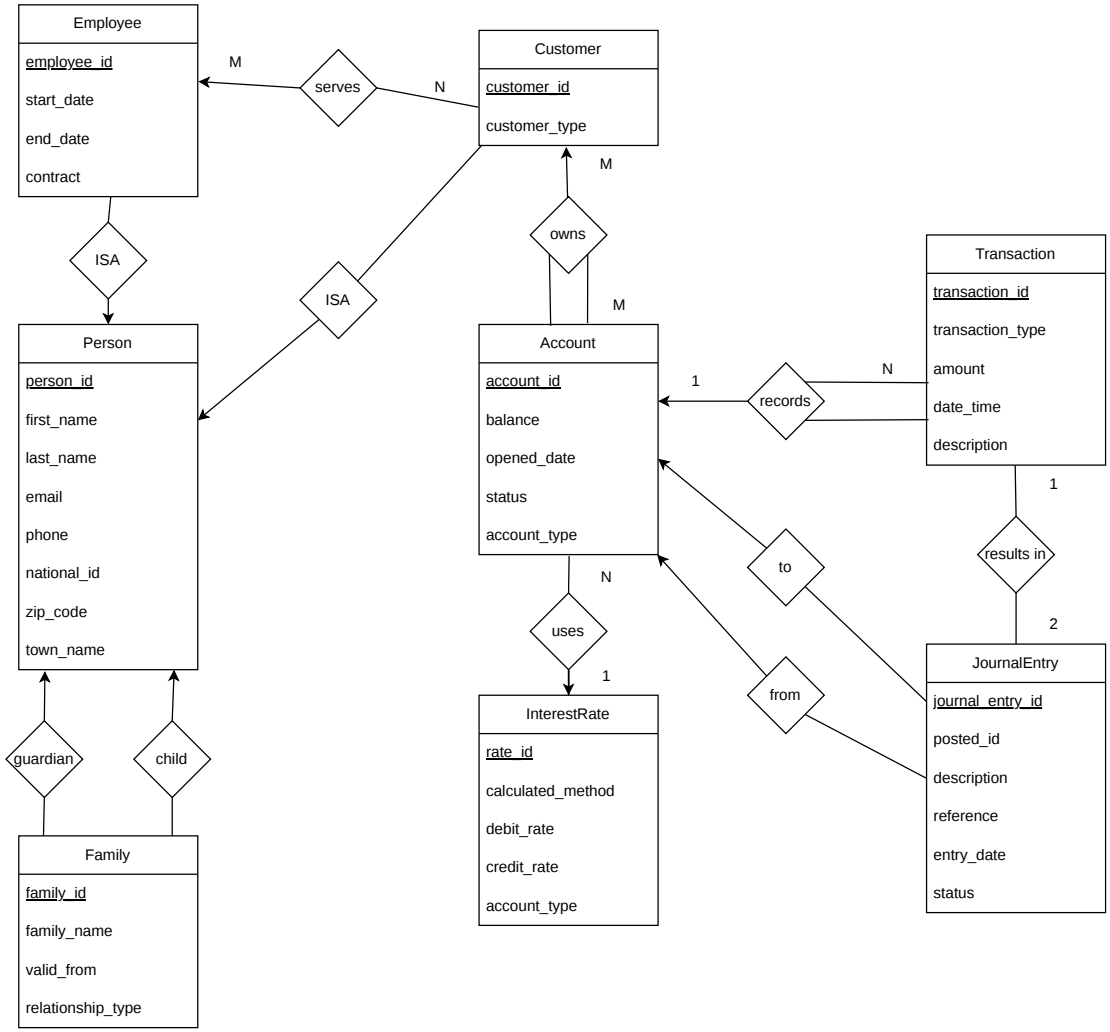

Figure A.5: Complete Entity-Relationship Diagram of the Banking System

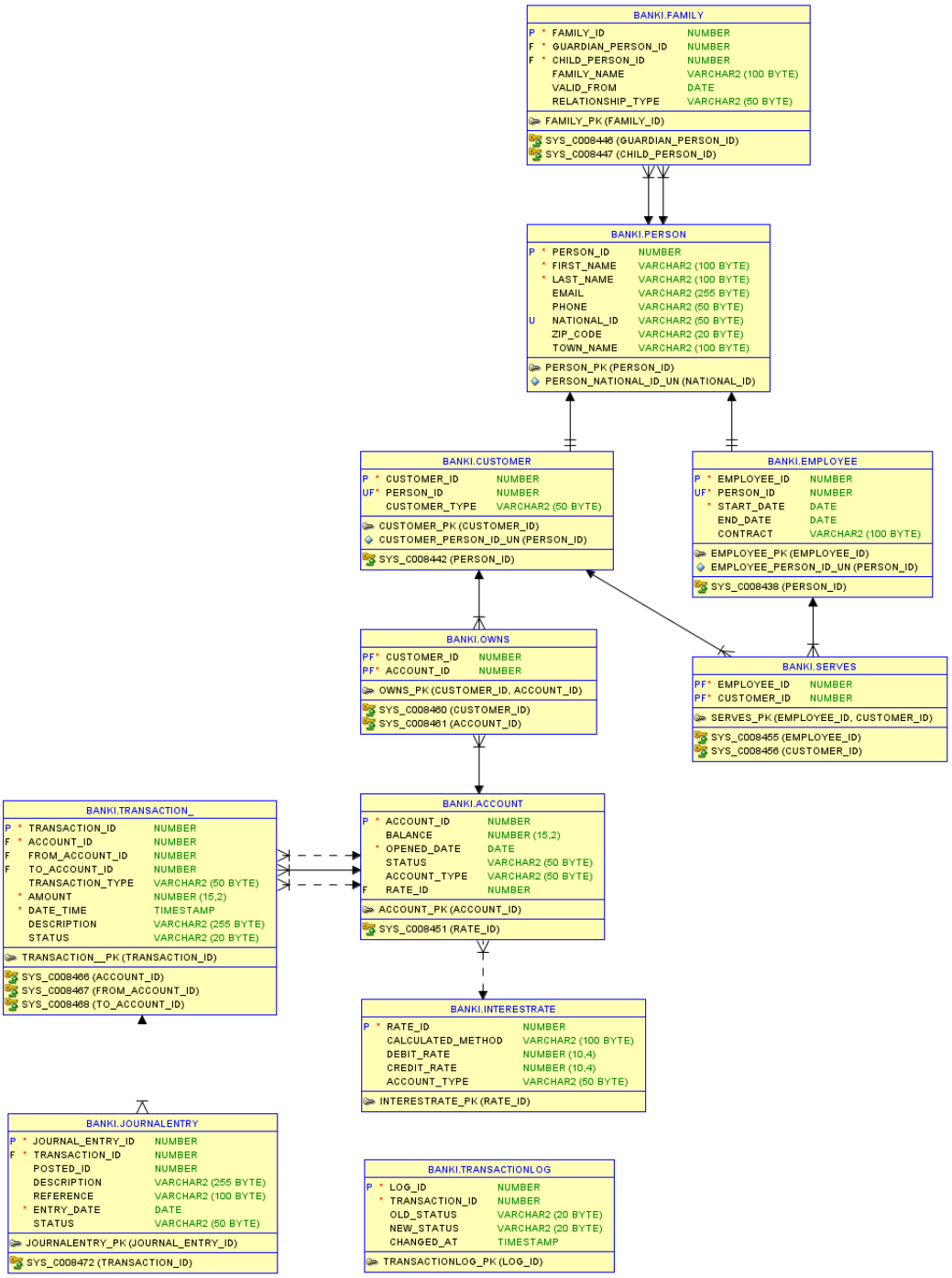

Figure A.6: Relational Schema of the E-R diagram

Bibliography

[1] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, 7th edition, 2020.